université PARIS-SACLAY
IUT DE CACHAN

Licence Pro. MECSE – Parcours SESAM

Programmation C

Prise en main de Qt (mode console)

Dép. GEii2

Auteur(s) : IUT Cachan

Date – Version : Octobre 2021 – v1.2

Référence : PROG_C_1 – Qt

PRESENTATION DE QT

Qt est un **Framework** (comme .Net ou SDL) sous la forme d'une **bibliothèque logicielle orientée objet** développée en **C++** par Qt Development Frameworks, filiale de Digia.

C'est un ensemble normalisé (API ①) de classes, de méthodes ou de fonctions qui sert de façade pour accéder aux services fournis.

Qt constitue une plateforme de **développement d'interfaces graphiques GUI** (*Graphical User Interface*) fournie à l'origine par la société norvégienne Troll Tech, rachetée par Nokia en février 2008 puis cédée intégralement en 2012 à Digia (www.qt.io).

Qt fournit également un ensemble de classes décrivant des éléments non graphiques :

- Accès aux données (fichier, base de données),
- Connexions réseaux (*socket*),
- Gestion du multitâche (*thread*),
- XML,
- etc.

Qt permet la **portabilité des applications** (qui n'utilisent que ses composants) par simple **recompilation du code source**.
Les environnements supportés sont :

- Les distributions **Unix** (dont **Linux**),
- **Windows,**
- **Mac OS X**.

De plus en plus de développeurs utilisent Qt, y compris parmi de grandes entreprises. On peut notamment citer : Google, Adobe Systems, Asus, Samsung, Philips, ou encore la NASA et bien évidemment Nokia.
Qt est notamment connu pour être la bibliothèque sur laquelle repose l'environnement graphique KDE, l'un des environnements de bureau les plus utilisés dans le monde Linux.

① Une API (Application Programming Interface) est une interface de programmation pour les applications et donc les développeurs d'applications !

Pour pouvoir développer des applications utilisant (ou non !) ce framework, nous avons besoin :

- D'un éditeur de texte pour créer le code source (taper les instructions du programme)
- Des outils pour générer l'exécutable à partir de ce code source (Compilateur, Editeur de liens)
- D'effectuer la mise au point du fonctionnement du programme (Débogage)

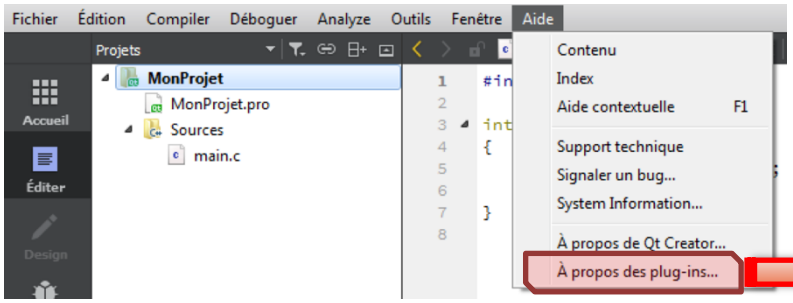
Nous avons donc besoin d'un IDE (Integrated Development Environment) qui regroupe tous ces outils : **QT Creator**

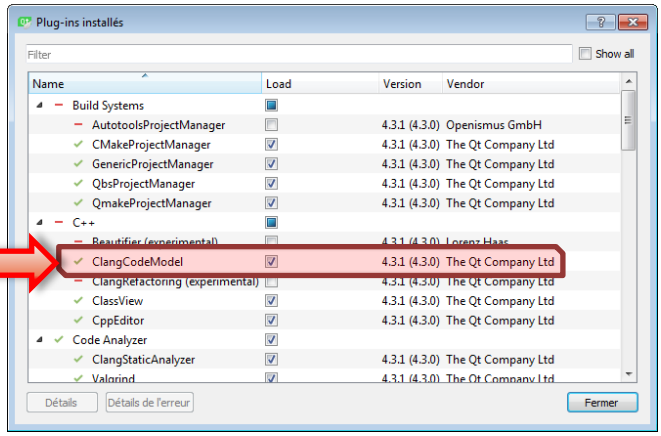
QT Creator est très intéressant pour le module car :

- Il est libre et open source
- Il est multiplateforme (version pour Windows ou pour Linux)
- Il est simple à installer
- Il est simple à prendre en main
- Il est performant
- Il permet de développer des applications allant du mode console à des IHM évoluées

CONFIGURATION

Pour que le débogueur affiche l'évolution du programme en mode pas à pas en C, et non en assembleur…





PREMIER PROGRAMME

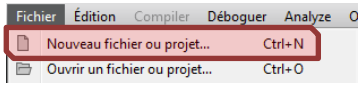
Créer un projet

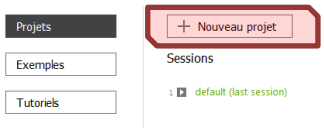
Un projet contient tous les éléments nécessaires pour compiler un programme et générer une application.

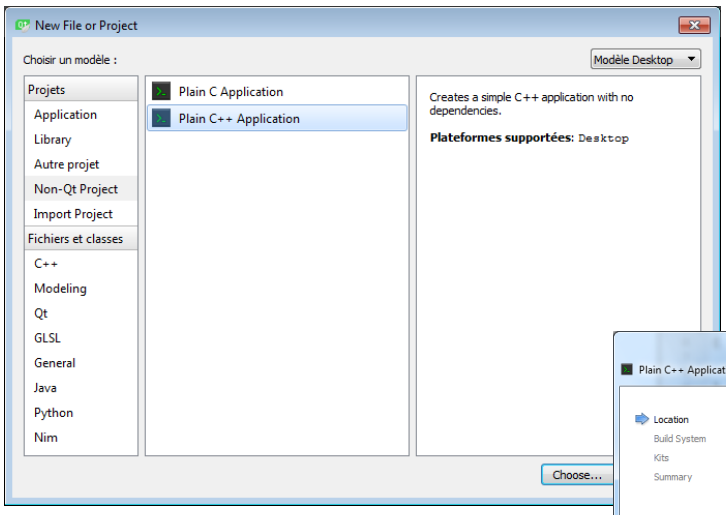
Lancer **qtcreator.exe** situé dans le répertoire **C:\Qt\Tools\QtCreator\bin**

Vous pouvez créer un nouveau projet :

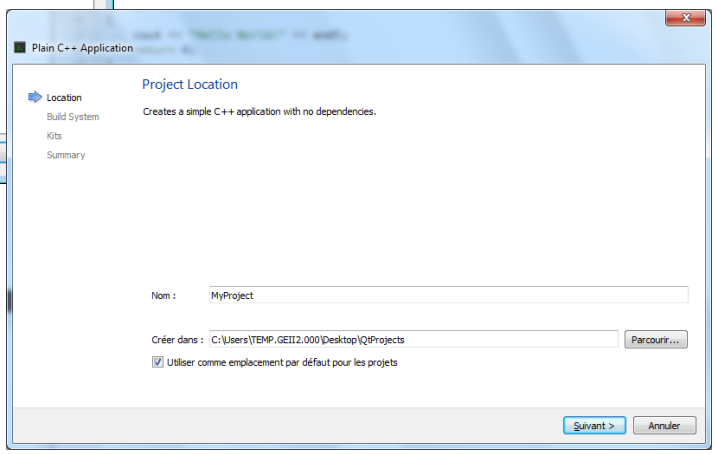
- Soit par le menu Fichier→ Nouveau fichier ou projet…
- Soit via la page d'accueil



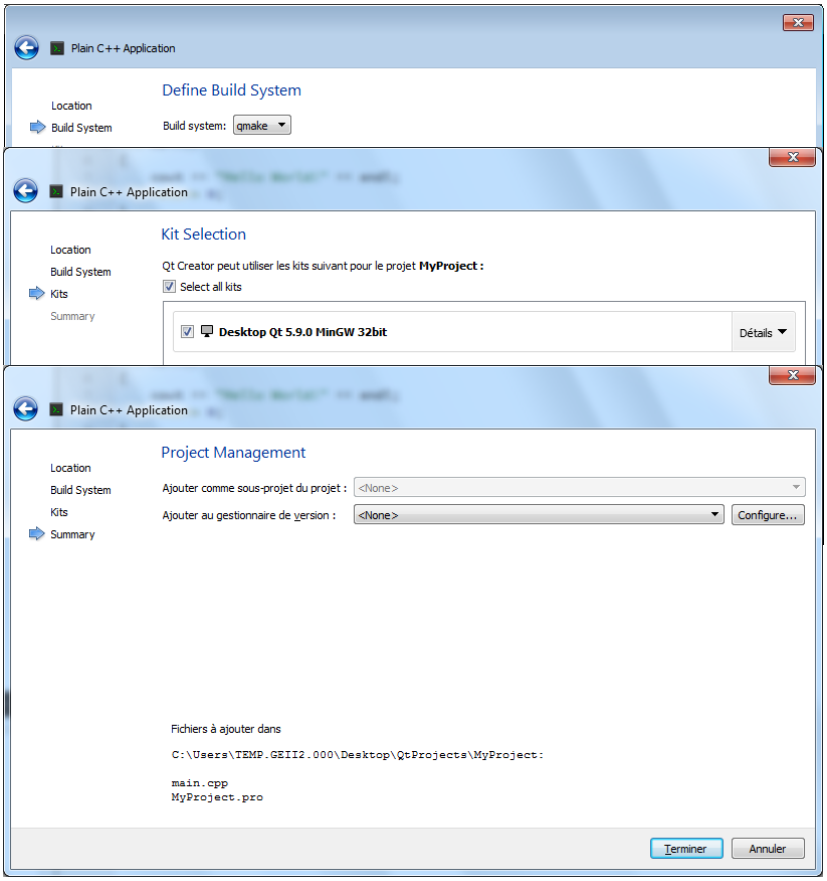




Une boîte de dialogue s'ouvre alors pour vous permettre de choisir le type de projet que vous souhaitez créer. Dans un premier temps, vous ne créerez que des projets du type ***Plain C++ Application*** dans les modèles ***Non-Qt Project***.



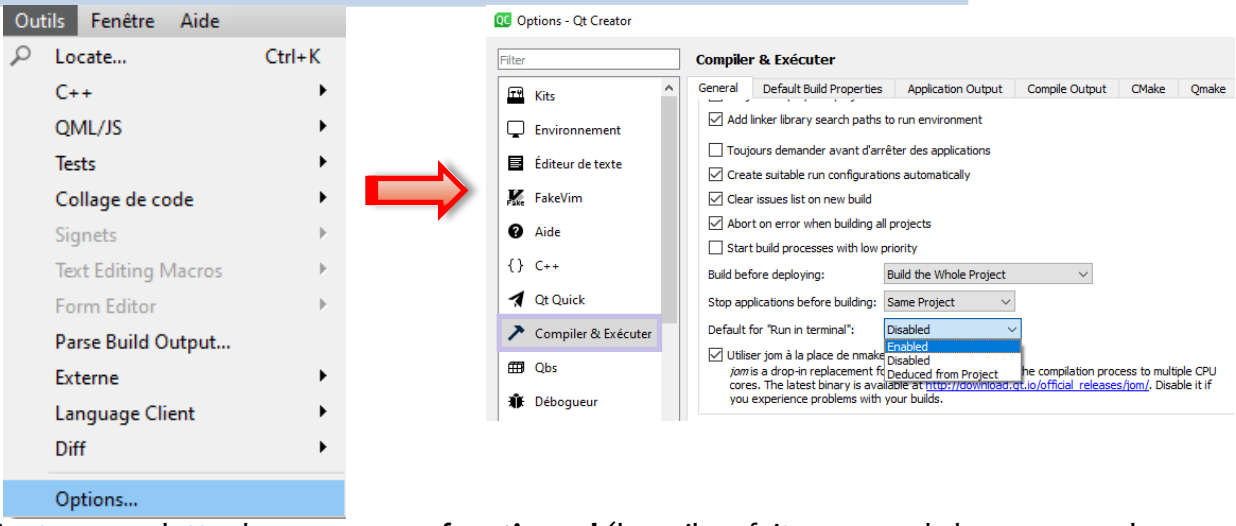
On peut alors choisir le nom et le répertoire de travail pour notre projet…



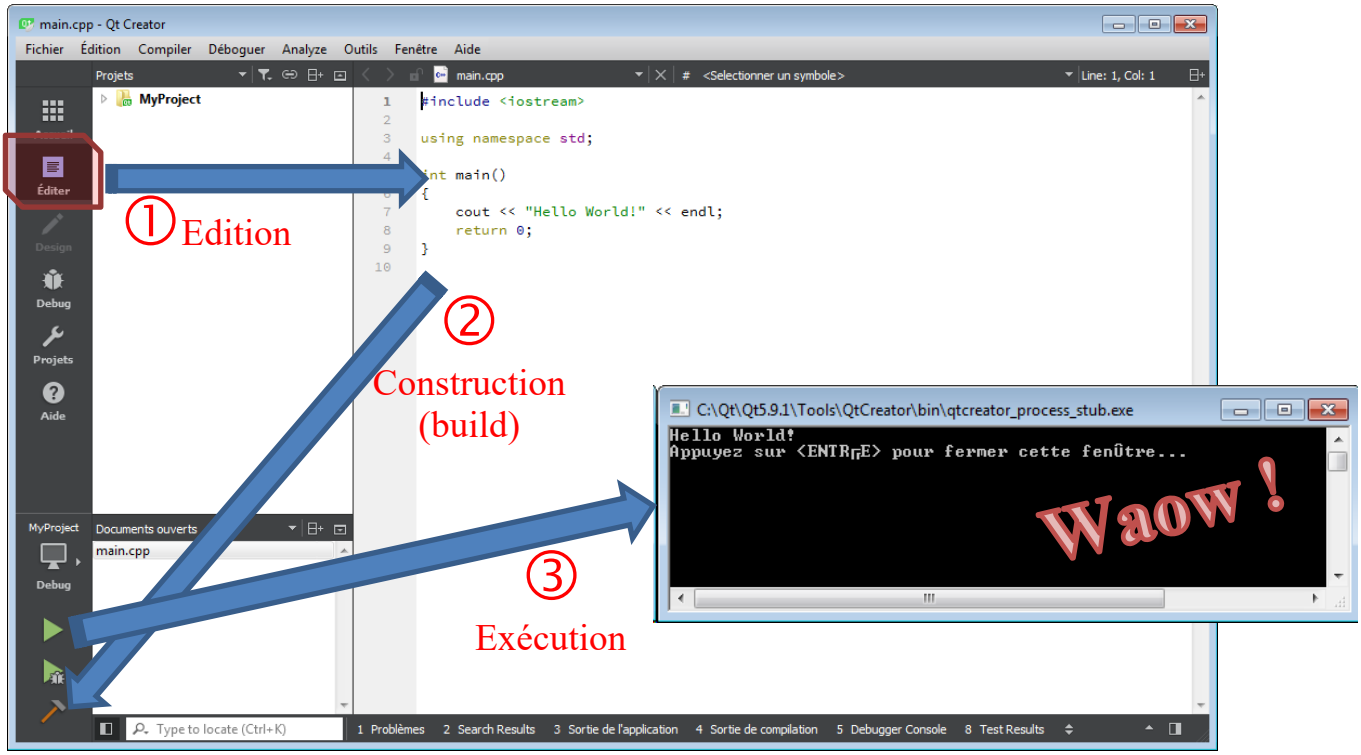
On va laisser les options par défaut pour les 3 fenêtres suivantes.

1. L'outil de construction du projet sera assuré par l'outil natif **qmake**
2. Il faut s'assurer que le kit ***Desktop Qt 5.15.xxx*** soit bien présent et coché.
3. On n'utilise pas (encore) ici de système de "versionning"

Dans le menu “outils/options”, aller dans la catégorie “Compiler & Exécuter”. S’assurer que ”Default for ‘Run in terminal’” est sur “Enabled”



On obtient un squelette de programme **fonctionnel** (bon, il ne fait pas grand-chose… pour le moment !)



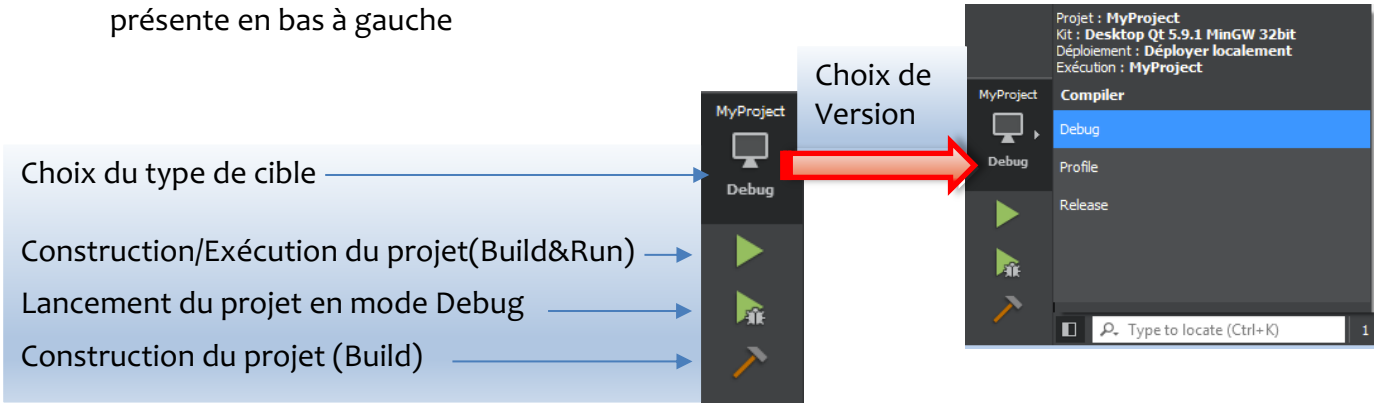
Remarque : **Afficher les caractères accentués dans la console sous Windows**

Pour que la console de Windows (la fenêtre noire d’exécution du programme) affiche correctement les caractères accentués, il faut que la « page de code » de la console et « la page de code » dans laquelle sont écrits les fichiers en langage C soient les mêmes. Pour dire les choses simplement, il faut que la console Windows et Qt parlent la même langue. Sous Qt, la « page de code » par défaut est WINDOWS-1252.
Pour forcer la même « page de code » dans la console de Windows, vous pouvez ajouter la ligne suivante au début de votre programme (au début du `main`) :

```
system("CHCP 1252 > nul");
```

“Construire” un projet (phase *Build*)

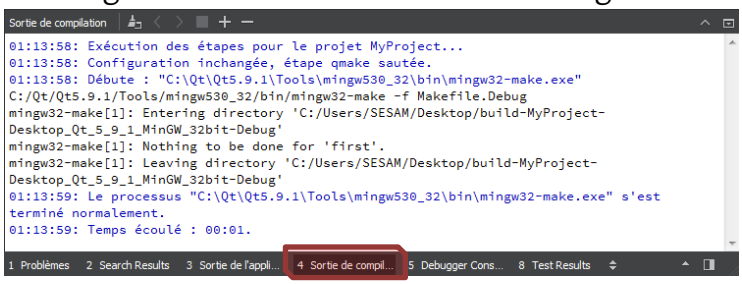
Vous pouvez éditer le fichier **main.c** en allant à gauche dans la fenêtre. Ce fichier comporte la fonction main, fonction principale et donc point d’entrée du programme.
Le fichier **main.c** contient les instructions pour afficher “Hello world !” à l'exécution. Pour pouvoir exécuter le programme, il faut tout d'abord construire (**build**) le projet. Cette opération peut être réalisée soit par le menu **Compiler**, soit par la barre d'outils **Compiler** présente en bas à gauche



La version finale (Release) donne un fichier exécutable plus petit et généralement plus efficace mais elle n'autorise pas le débogage. Les fichiers sources compilés sont rangés dans le sous-répertoire obj du projet. Le programme exécutable se trouve dans le sous-répertoire ..\build-NomDuProjet-Desktop_Qt_5_xxxxxx_Debug pour la version de débogage (Debug) et dans le sous-répertoire ..\build-NomDuProjet-Desktop_Qt_5_xxxxxxxx-Release pour la version finale.


On travaillera essentiellement en mode **Debug** pour nos manipulations.

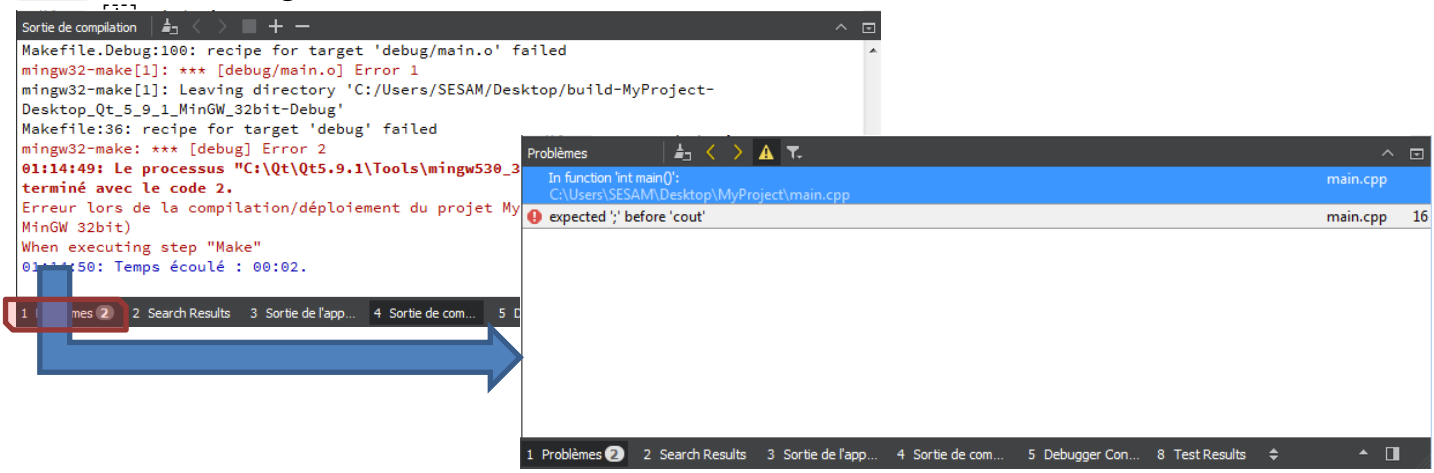
On obtient l'affichage du résultat d'un “**Build**” dans l'onglet en bas : “**4 Sortie de compilation**”.



Erreur à la compilation :

Les erreurs lors de la compilation du projet sont recensées dans l'onglet “**1 Problèmes**”.
Un simple clic sur l'erreur envoie le curseur dans le code à l'endroit où l'erreur est détectée. Un marqueur rouge dans la marge du code indique la ligne concernée.


Attention, souvenez-vous qu'en langage C, un point-virgule oublié en fin de ligne entraîne une erreur sur la ligne suivante.

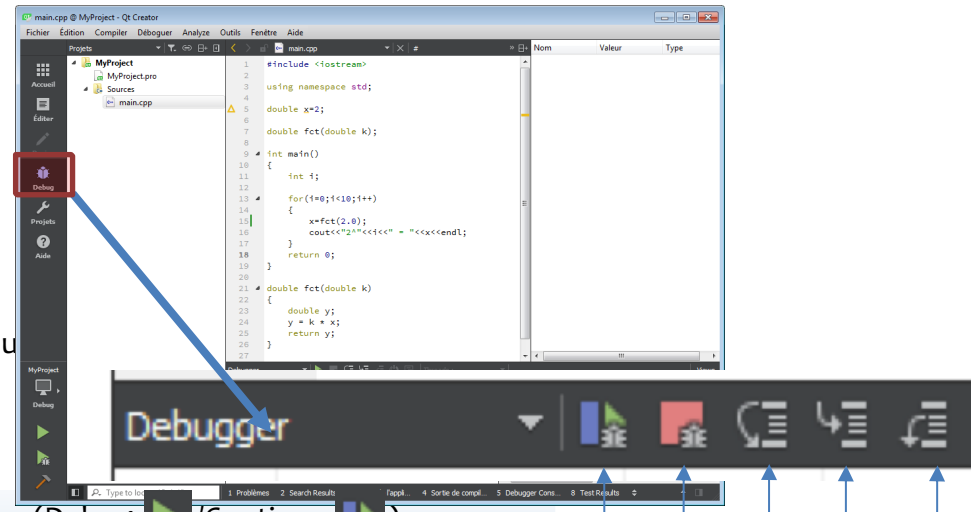


on peut lancer le programme (**Run**) :
Une fenêtre console s'ouvre et affiche le résultat de l'exécution.
Mais que fait-on si le fonctionnement n'est PAS celui attendu … ?

Mise au point d'un projet (mode *Debug*)

Il est possible d'utiliser le débogueur soit à partir du menu **Déboguer**, soit grâce à la barre d'outils **Debugger**.

La barre d'outils Debu

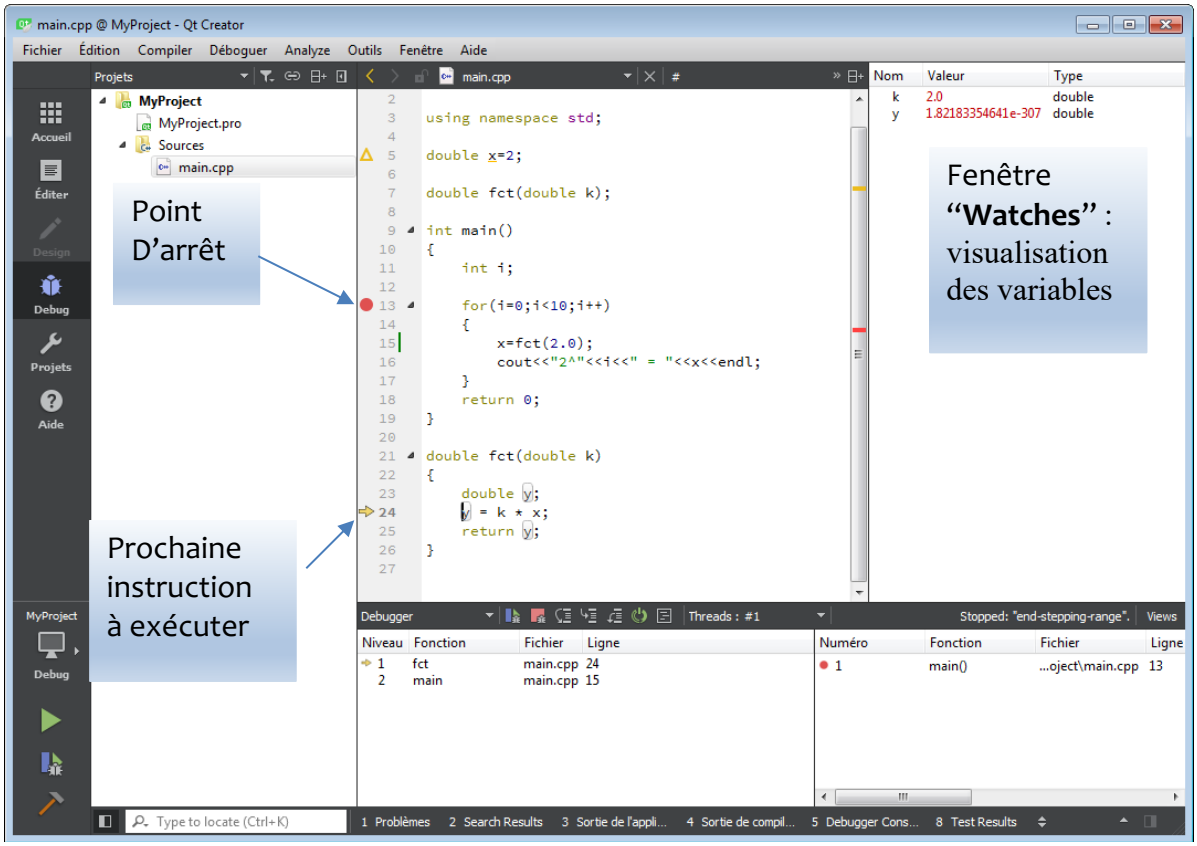


Exécute en mode débogueur (Debug /Continue)
⚠ Nécessite de placer un ou des points d'arrêts (breakpoints)
Arrêter le mode **Debug**
Exécute jusqu'au début de la fonction main OU la ligne suivante
⚠ Sans entrer dans les fonctions
Exécute une ligne en entrant dans les fonctions (Step In)
Exécute jusqu'à la fin de la fonction en cours (Step out)

Le principe consiste à exécuter le programme pas à pas, depuis le début ou à partir d'un point particulier (mise en place d'un point d'arrêt : **Breakpoint**)

- Vous pouvez placer des points d'arrêt en cliquant à gauche du numéro de la ligne où vous voulez vous arrêter.

Pour chaque étape exécutée, on doit pouvoir **visualiser l'état des variables** et leur évolution.



Dorénavant, vous avez tous les outils pour :

- Créer un projet
- Editer un programme
- Construire l'application correspondante
- Déboguer une application

A vous de jouer !